

jQuery

Les bibliothèques et les Frameworks JavaScript sont efficaces pour accélérer le processus de développement de votre site web ou de votre application.

C'est quoi jQuery

jQuery est une bibliothèque JavaScript très populaire qui simplifie considérablement la manipulation du Document Object Model (DOM), la gestion des événements et la création d'animations sur les pages web. Elle a été créée en 2006 par John Resig au BarCamp NYC. JQuery est un logiciel gratuit et open source avec une licence MIT.

Pourquoi utiliser jQuery ?

- **Simplification du code** : Réduire la quantité de code nécessaire pour réaliser des tâches courantes (sélectionner des éléments, modifier leur contenu ou gérer des événements...)
- **Compatibilité multi-navigateurs** : Ecrire du code qui fonctionne de manière cohérente sur tous les navigateurs modernes.
- **Grande communauté** : Trouver facilement de la documentation et de l'aide en ligne grâce à une communauté très active.
- **Plugins** : Il existe une multitude de plugins jQuery qui étendent ses fonctionnalités, vous permettant de réaliser des effets visuels complexes, des interactions avancées avec l'utilisateur, et bien plus encore.

Ajouter jQuery à vos pages Web

Il existe plusieurs façons d'utiliser jQuery sur votre site Web. Vous pouvez :

- Téléchargez la bibliothèque jQuery (un fichier JavaScript unique) depuis <https://jquery.com/download/> et vous la référencez avec la balise <script> à l'intérieur de la section <head>

```
<head>
<script src="jquery-3.7.1.min.js"></script>
</head>
```

- Inclure jQuery à partir d'un CDN (Content Delivery Network), comme Google.

```
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
</head>
```

Syntaxe jQuery

La syntaxe de base est : **\$(selector). action ()**

- Un signe \$ pour définir/accéder à jQuery
- Un (sélecteur) pour « interroger (ou trouver) » des éléments HTML
- Une action jQuery () à effectuer sur le(s) élément(s)

Les sélecteurs jQuery

jQuery utilise des sélecteurs similaires à ceux de CSS pour sélectionner des éléments HTML. Tous les sélecteurs dans jQuery commencent par le signe dollar et les parenthèses : \$().

Sélecteur d'éléments	Sélectionner les éléments en fonction du nom de l'élément.	\$("p")
Sélecteur #id	Utiliser l'attribut id d'une balise HTML pour trouver l'élément spécifique.	\$("#monElement")
Sélecteur .class	Trouver des éléments avec une classe spécifique.	\$(".maClass")
Autres sélecteurs	Sélectionner les éléments en fonction du sélecteur css.	\$("p.intro") \$("ul li:first") \$(":button") \$("a[target!='_blank'] ») \$("tr:even")

Méthodes d'événements jQuery

Un événement représente le moment précis où quelque chose se produit.

Exemples :

- déplacer une souris sur un élément,
- sélection d'un bouton radio,
- en cliquant sur un élément,
- ...

Dans jQuery, la plupart des événements DOM ont une méthode jQuery équivalente qui facilite la gestion des événements comme les clics, les survols, etc.

Ci-dessous quelques méthodes d'événements jQuery couramment utilisées :

<code>\$(document).ready()</code>	permet d'exécuter une fonction lorsque le document est entièrement chargé.	
<code>click()</code>	La fonction est exécutée lorsqu'un événement de clic se déclenche sur l'élément HTML.	<code>\$("#div").click(function(){ // l'action });</code>
<code>dblclick()</code>	La fonction est exécutée lorsque l'utilisateur double-clique sur l'élément HTML	<code>\$("#div1").dblclick(function(){ \$(this).hide(); });</code>
<code>mouseenter()</code>	La fonction est exécutée lorsque le pointeur de la souris entre dans l'élément HTML	<code>\$("#div1").mouseenter(function(){ // l'action });</code>
<code>mouseleave()</code>	Exécuter une fonction lorsque le pointeur de la souris quitte l'élément HTML	<code>\$("#div1").mouseleave(function(){ // l'action });</code>
<code>mouseup()</code>	La fonction est exécutée lorsque le bouton de la souris est relâché, alors que la souris se trouve sur l'élément HTML.	<code>\$("#div1").mouseup(function(){ // l'action });</code>

Les effets jQuery

jQuery offre un ensemble d'outils pour créer des animations fluides et personnalisées.

Syntaxe	Description
<code>\$(selector).hide(speed,callback);</code>	Masquer des éléments HTML
<code>\$(selector).show(speed,callback);</code>	Afficher des éléments HTML
<code>\$(selector).toggle(speed,callback);</code>	Basculer entre masquer et afficher des éléments HTML
<code>\$(selector).fadeIn(speed,callback);</code>	Apparaître un élément caché en fondu
<code>\$(selector).fadeOut(speed,callback);</code>	Estomper un élément visible.
<code>\$(selector).fadeToggle(speed,callback);</code>	Bascule entre les méthodes
<code>\$(selector).fadeTo(speed,opacity,callback);</code>	Réaliser un fondu vers une opacité donnée
<code>\$(selector).animate({params},speed,callback);</code>	Ajouter des animations personnalisées à vos éléments HTML
<code>\$(selector).stop(stopAll,goToEnd);</code>	Arrêter les animations ou les effets avant qu'ils ne soient terminés.

Note :

- Le paramètre requis (**params**) définit les propriétés CSS à manipuler.
- Tous les noms de propriétés CSS doivent être en **camel-case** lorsqu'ils sont utilisés avec la méthode `animate()` : `paddingLeft` au lieu de `padding-left`, `marginRight` au lieu de `margin-right`, etc.
- Pour manipuler la position, vous devez définir la propriété CSS **position** de l'élément sur **relative**, **fixed** ou **absolue**.
- Les paramètres `speed`, `callback`, `stopAll` et `goToEnd` sont facultatifs.
- La valeur par défaut de **stopAll** est **false**, ce qui signifie que seule l'animation active sera arrêtée, ce qui permettra à toutes les animations en file d'attente d'être exécutées par la suite.
- La valeur par défaut du paramètre **goToEnd** est **false**, spécifie si l'animation en cours doit être terminée immédiatement ou non.
- En mettant `+=` ou `-=` devant la valeur courante de l'élément cela permet définir des valeurs relatives
- Réaliser différentes animations les unes après les autres (animations en file d'attente) en faisant plusieurs appels à la méthode `animate()`

La Fonction de rappel (Callback)

- Les instructions JavaScript sont exécutées ligne par ligne. Cependant, avec les effets, la ligne de code suivante peut être exécutée même si l'effet n'est pas terminé. Cela peut créer des erreurs.
- Pour éviter cela, vous pouvez créer une fonction de rappel, cette dernière est exécutée une fois l'effet actuel terminé.

Le chaînage

Le chaînage permet d'exécuter **plusieurs méthodes** jQuery **sur le même élément** dans **une seule instruction**.

```
$("#id").css("color", "red").slideUp(2000).slideDown(2000);
```

Vous pouvez formater la ligne de code qui est assez longues en ajoutant les sauts de ligne et les indentations.

Manipuler les éléments et les attributs HTML

jQuery est livré avec un ensemble de méthodes liées au DOM (**Document Object Model**) qui facilitent l'accès et la manipulation des éléments et des attributs.

text()	Définit ou renvoie le contenu textuel des éléments sélectionnés
html()	Définit ou renvoie le contenu des éléments sélectionnés (y compris le balisage HTML)
val()	Définit ou renvoie la valeur des champs de formulaire
append()	Insère du contenu à la fin des éléments sélectionnés
prepend()	Insère du contenu au début des éléments sélectionnés
after()	Insère du contenu après les éléments sélectionnés
before()	Insère du contenu avant les éléments sélectionnés
remove()	Supprime l'élément sélectionné (et ses éléments enfants), cette méthode accepte également un paramètre, qui permet de filtrer les éléments à supprimer.
empty()	Supprime les éléments enfants de l'élément sélectionné

Remarque :

- Les méthodes after() et before() peuvent prendre un **nombre infini** de nouveaux éléments comme paramètres.
- Les nouveaux éléments peuvent être générés avec **du texte/HTML** (comme nous l'avons fait dans l'exemple ci-dessus), avec **jQuery** ou avec **du code JavaScript et des éléments DOM**.

Manipuler les CSS

jQuery dispose de plusieurs méthodes pour manipuler les CSS.

css("propertyname");	Pour renvoyer la valeur d'une propriété CSS spécifiée	\$("#div").css("background-color");
css("propertyname", "value");	Pour définir une propriété CSS spécifiée	\$("#div").css("background-color", "yellow");
addClass()	Ajoute une ou plusieurs classes aux éléments sélectionnés	\$("#h1, h2, p").addClass("blue"); \$("#div").addClass("important");
removeClass()	Supprime une ou plusieurs classes des éléments sélectionnés	\$("#h1, h2, p").removeClass("blue");
toggleClass()	Bascule entre l'ajout/la suppression de classes des éléments sélectionnés	\$("#h1, h2, p").toggleClass("blue");

A J A X (Asynchronous JavaScript and XML)

C'est quoi AJAX ?

AJAX **n'est pas un langage de programmation** mais correspond plutôt à un **ensemble de techniques utilisant des technologies diverses** qui permettent de créer des interfaces utilisateur dynamiques et réactives.

Il permet **d'envoyer et de récupérer** des données **vers et depuis un serveur** de façon **asynchrone**, c'est-à-dire sans avoir à recharger la page.

La première formalisation du terme AJAX date de 2005 (par Jesse James Garrett un consultant en interface utilisateur) mais les techniques utilisées ont commencé à être mises en place dès la fin des années 1990.

A sa création, **AJAX** utilisait les technologies suivantes qui lui ont donné son nom :

- Le **XML**, le format de données le plus souvent utilisé pour échanger des informations entre le navigateur et le serveur. Il permet de structurer les données de manière lisible par les machines;
- Le **JavaScript**, le cœur d'Ajax. C'est lui qui permet de manipuler le contenu d'une page web, de faire des requêtes vers un serveur et de mettre à jour le contenu en fonction des réponses reçues ;
- L'**objet XMLHttpRequest** intégré au navigateur pour la communication asynchrone, les requêtes faites au serveur ne bloquent pas l'exécution du code JavaScript, cela signifie que l'utilisateur peut continuer à interagir avec la page pendant que la requête est en cours de traitement en arrière-plan;
- Le **HTML** et le **CSS** pour la présentation des données.

Remarque :

Ajax est aujourd'hui un terme générique utilisé pour désigner toute technique côté client (côté navigateur) permettant d'envoyer et de récupérer des données depuis un serveur et de mettre à jour dynamiquement le DOM sans nécessiter l'actualisation complète de la page.

Pourquoi utiliser Ajax ?

Aujourd'hui, Ajax est omniprésent dans les applications web modernes. Il est difficile d'imaginer une application web complexe sans utiliser Ajax.

▪ **Expérience utilisateur améliorée :**

Les pages web deviennent plus réactives, les informations sont mises à jour en temps réel, sans que l'utilisateur ait à attendre le rechargement complet de la page.

▪ **Applications web riches :**

Ajax permet de créer des applications web qui se comportent presque comme des applications de bureau, avec des fonctionnalités avancées comme le glisser-déposer, les autocomplétions, etc.

▪ **Moins de bande passante :**

En ne rechargeant que les éléments qui ont besoin d'être mis à jour, Ajax réduit la quantité de données à transférer entre le navigateur et le serveur.

Exemples d'utilisation d'Ajax:

- Les boîtes de recherche suggérant des résultats au fur et à mesure de la saisie.
- Les paniers d'achat qui se mettent à jour en temps réel.
- Les réseaux sociaux qui affichent les nouvelles notifications sans recharger la page.
- Les applications de messagerie instantanée.

Note :

- **Ajax est une technologie clé pour créer des interfaces web modernes et performantes.**
- Elle est largement utilisée dans le développement web, et de nombreux frameworks JavaScript (comme React, Angular ou Vue.js) s'appuient sur ces principes pour faciliter la création d'applications web complexes.

L'évolution d'Ajax :

- **JSON:** Bien que XML ait été initialement utilisé pour échanger des données, JSON (JavaScript Object Notation) est rapidement devenu le format de données préféré pour Ajax, en raison de sa simplicité et de sa légèreté.
- **Les frameworks JavaScript:** Des bibliothèques et de frameworks comme jQuery, Angular, React et Vue.js ont simplifié l'utilisation d'Ajax en fournissant des abstractions et des outils pour gérer les requêtes asynchrones, la manipulation du DOM, etc.
- **Les API REST:** Les API REST (Representational State Transfer) sont devenues le standard pour concevoir des interfaces de programmation pour les applications web, et Ajax est le moyen privilégié pour consommer ces API.

jQuery et AJAX

Le tableau suivant répertorie quelques méthodes jQuery AJAX

méthode	Description
<code>\$.ajax({name:value, name:value, ... })</code>	<code>ajax()</code> est utilisée pour effectuer une requête AJAX (HTTP asynchrone).
<code>\$.ajaxSetup({name:value, name:value, ... })</code>	<code>ajaxSetup()</code> définit les valeurs par défaut pour les futures requêtes AJAX.
<code>\$.get(URL,data,function(data,status,xhr),dataType)</code>	<code>\$.get()</code> charge les données du serveur à l'aide d'une requête HTTP GET.
<code>\$(selector).getJSON(url,data,success(data,status,xhr))</code>	<code>getJSON()</code> est utilisée pour obtenir des données JSON à l'aide d'une requête HTTP GET AJAX.
<code>\$(selector).getScript(url,success(response,status))</code>	<code>getScript()</code> est utilisée pour obtenir et exécuter un script JavaScript à l'aide d'une requête HTTP GET AJAX.
<code>\$(selector).post(URL,data,function(data,status,xhr),dataType)</code>	<code>\$.post()</code> charge les données du serveur à l'aide d'une requête HTTP POST.
<code>\$(document).ajaxComplete(function(event,xhr,options))</code>	<code>ajaxComplete()</code> spécifie une fonction à exécuter lorsqu'une requête AJAX est terminée.